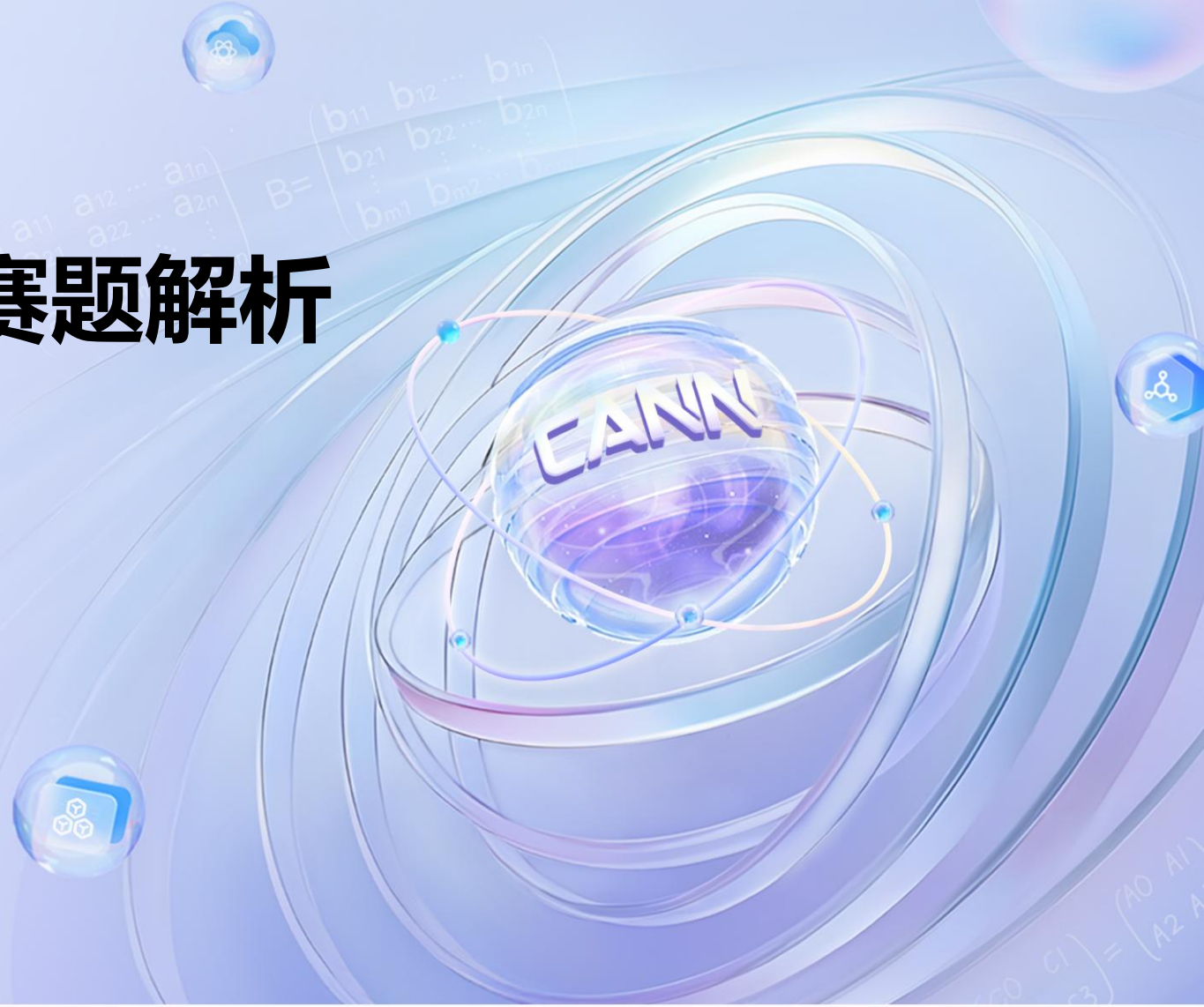
# CANN 挑战赛决赛赛题解析

**分享嘉宾：之卓**

**时间：**2026年10月






CANN

# 目录

Part 1 赛程安排

Part 2 赛题解读

Part 3 交流答疑

# 大赛简介及日程

**CANN挑战赛**是由 CANN开源社区 主办的赛事。大赛依托华为算法挑战赛，围绕昇腾AI算子开发、功能实现与性能优化设置竞赛任务，并提供配套学习资料和在线实践环境，帮助参赛选手了解算子开发流程，提升编程实践和性能优化能力。本届赛事设置 **杭厦、上合、京津东北、西北、西南**五大赛区，面向全国相关区域高校学生开放报名。欢迎高校学子参与挑战。

当前进度

## 初赛

在线答题

| 7/21 | 9/5 | 10/16 | 10/17 (周六) | 10/18 | 10/19 |
|---|---|---|---|---|---|
| 启动报名 | 赛题发布<br>启动初赛 | 报名截止 | 线上初赛截止 | 公示 | 晋级名单公布 |

**前32强队伍晋级复赛**

## 决赛

**第一批 | 京津东北 · 西南 · 西北**

| 10/23 | 11/6 | 11/7（周六） |
|---|---|---|
| 赛题发布 + 直播解读 | 外地选手到场 | **现场决赛&颁奖** |

**第二批 | 上合 · 杭厦**

| 10/30 | 11/13 | 11/14（周六） |
|---|---|---|
| 赛题发布 + 直播解读 | 外地选手到场 | **现场决赛&颁奖** |

外地选手11/13到场；本地选手11/14到场

# 目录

Part 1 赛程安排

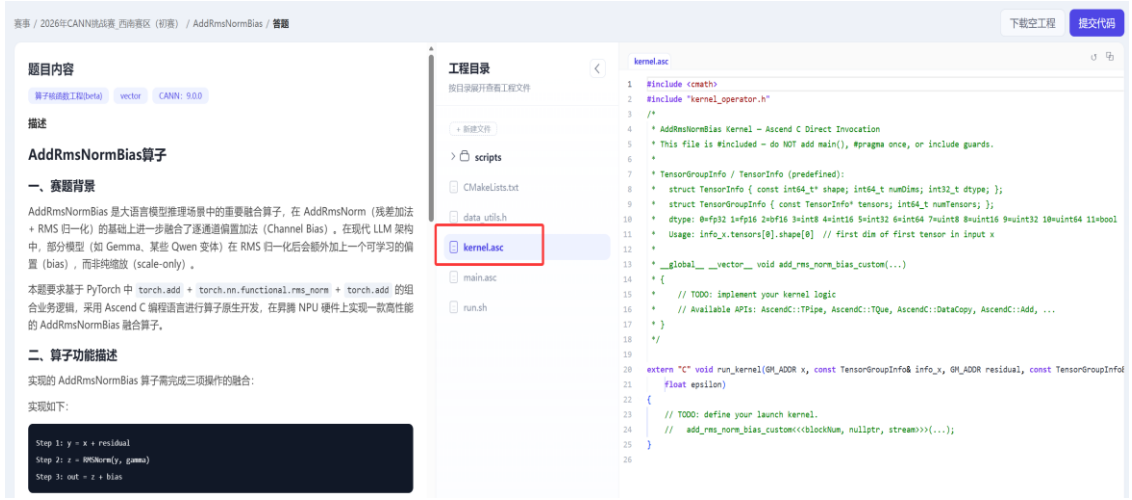
Part 2 赛题解读
（杭厦、上合）

Part 3 交流答疑

# 赛题解读-判题过程与得分介绍

## 判题过程介绍

- 完成赛题要求的一个文件并点击提交代码。




- 平台会自动跳转到提交界面，后台Case验证完成后返回结果。




*注意：编译错误可在执行结果界面查看具体报错*

## 得分规则

- 提交完成后可查看提交排名。




- 本次比赛共15个测试点，所有case点精度全部通过才会计分。
- 每个测试点单独计分，逻辑如下（T为最优性能，t为当前提交性能）：

$$\frac{100}{1+\log_{1.5}\frac{t}{T}}$$

- 排行榜显示的最终分数为所有case得分的均值。若得分计算一致，则以提交时间进行排序，提交越早，排名越高。

*注意：排行榜仅针对最后一次提交呈现，请确保比赛结束前提交的是最优实现*

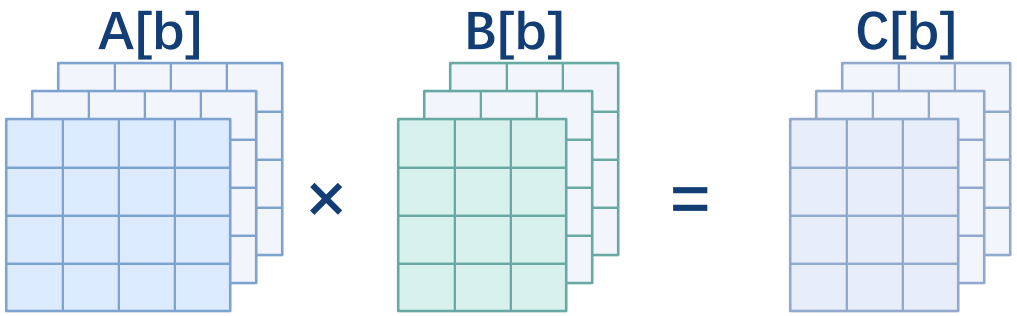
CANN

# 杭厦上合决赛赛题解读-Grouped Matmul介绍

## Batch MatMul

等大矩阵按 batch 堆叠

**批次维 B = 3**

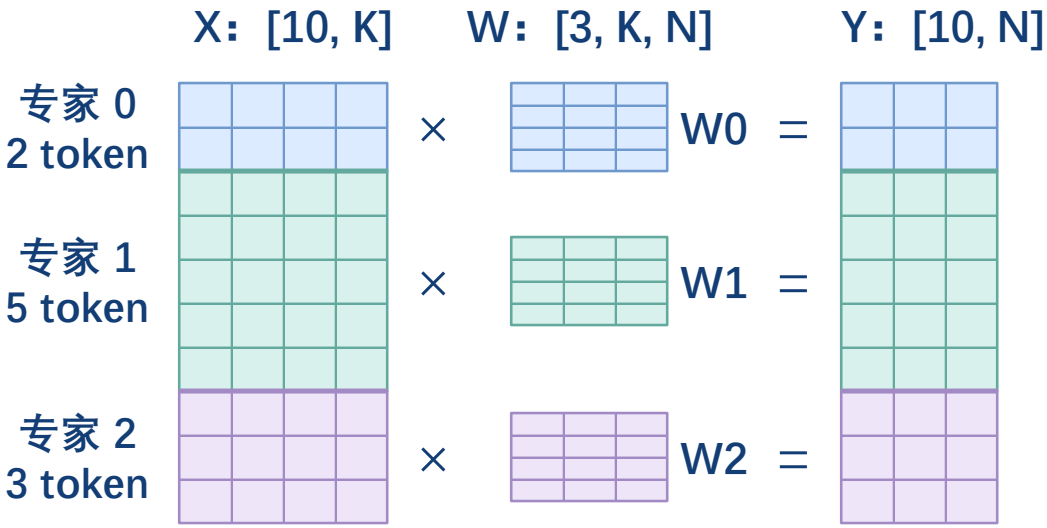



$$C[b] = A[b] \times B[b]$$

b = 0, 1, 2，各批次的 M、K、N 相同

## Grouped MatMul

token 按专家分段，组长可以不同




**groupList = [2, 7, 10]**

累计结束位置，对应组长 [2, 5, 3]

输入与输出保留相同的专家分组边界



CANN

# （1/3）杭厦决赛赛题解读-赛题背景

## ■ 赛题背景：

① 在 MoE 专家网络中，token 经路由后分配到不同专家，并按专家连续排列。
专家之间的 token 数往往不同，逐组执行矩阵乘容易产生额外启动开销和负载不均。

② GMM 将这些分组线性计算统一调度；本题采用 INT8 激活与权重，
通过逐 token、逐通道的量化因子还原浮点结果，用于后续网络计算。

③ 专家任务的规模由运行时分组决定，单个专家可能只接收少量 token，也可能为空。
同时，输入按全局 token 行排列，权重与通道量化因子按专家组织，需要保持索引对应。

**核心要点**

- 负载不均衡：每次matmul做计算的专家数不一致。
- 非对齐尾块：对于非对齐场景，需要考虑如何正确处理。
- cv并行处理：考虑cube，vector核怎么并行处理。

**路由并按专家排列 token**
**各专家的 token 数不同**

→

**INT8 分组矩阵乘 + 反量化**
**本题计算范围**

→

**FP16 / BF16 输出**
**交给后续网络计算**

## ■ 赛题特点

**① 本次比赛要求使用 Ascend C 编程语言完成 Groupmatmul 算子开发**
**② 在满足精度的基础上，比拼算子性能：负载不均衡，cv并行协同，非对齐处理**

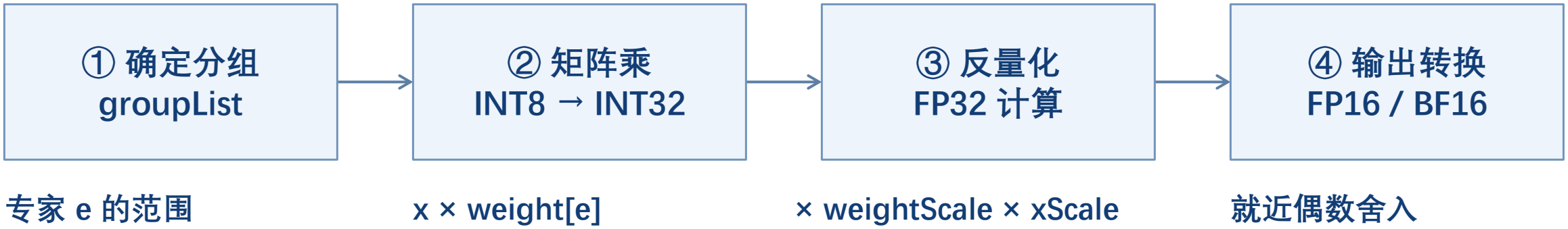


CANN

# （2/3）杭厦决赛赛题解读-题目概述

## ■ 题目概述：

实现 Groupedmatmul 算子需完成：依据grouplist分组->矩阵乘法->反量化->输出转化




## ■ 算子功能计算公式

$$acc[m,n] = \sum_k x[m,k] \times weight[e,k,n]$$

$$y[m,n] = cast(float(acc) \times float(weightScale[e,n]) \times xScale[m])$$

## ■ 分组属性：

groupListType=0：累计结束位置；=1：各专家 token 数
例：[1,4,9] 与 [1,3,5] 对应相同分组；m 始终使用全局行下标

CANN

# （3/3）杭厦决赛赛题解读-约束说明

| 参数名 | 类型 | 形状维度 | 支持数据类型 | 数据格式 | 备注 |
| --- | --- | --- | --- | --- | --- |
| x | 输入 | (M, K) | INT8 | ND | 按专家排列的 token 激活 |
| weight | 输入 | (E, K, N) | INT8 | ND | 每个专家一组权重，不转置 |
| xScale | 输入 | (M,) | FP32 | ND | 每个 token 的反量化因子 |
| weightScale | 输入 | (E, N) | FP32 / BF16 | ND | 每个专家、每个输出通道的因子 |
| groupList | 输入 | (E,) | INT64 | ND | 累计结束位置或各专家 token 数 |
| y | 输出 | (M, N) | FP16 / BF16 | ND | 保持输入 token 的行顺序 |
| groupListType | 属性 | 标量 | INT64 | — | 0：累计结束；1：各组 token 数 |
| outputDtype | 属性 | 标量 | string | — | float16 或 bfloat16 |

CANN

# （1/3）上合决赛赛题解读-赛题背景

## ■ 赛题背景：

①在多专家线性层中，各专家接收的 token 数不同，分组矩阵乘将这些变长任务统一组织。
采用 INT8 激活与权重可以降低存储和搬运开销，计算后还需反量化并执行 GELU 激活。

② GroupedMatmulGelu 将矩阵乘、反量化与 GELU 融合为一次设备计算，
减少中间矩阵的 GM 读写和额外 Kernel 调用，用于多专家网络中的线性层与激活计算。

③每个专家使用独立权重与逐通道 scale，后者由调用方预先合并激活与权重量化系数。
运行时分组可能包含空专家和非对齐尾块，需保持 token 区间、权重与 scale 的正确对应。

**核心要点**

- 负载不均衡：每次matmul做计算的专家数不一致。
- 非对齐尾块：对于非对齐场景，需要考虑如何正确处理。
- cv并行处理：考虑cube，vector核怎么并行处理。

**按专家排列的 INT8 token**
**各组 token 数可变**

→

**INT8 分组矩阵乘**
**逐通道反量化**

→

**tanh GELU 激活**
**输出 FP16**

## ■ 赛题特点

**① 本次比赛要求使用 Ascend C 编程语言完成 Groupmatmul 算子开发**
**② 在满足精度的基础上，比拼算子性能：负载不均衡，cv并行协同，非对齐处理**

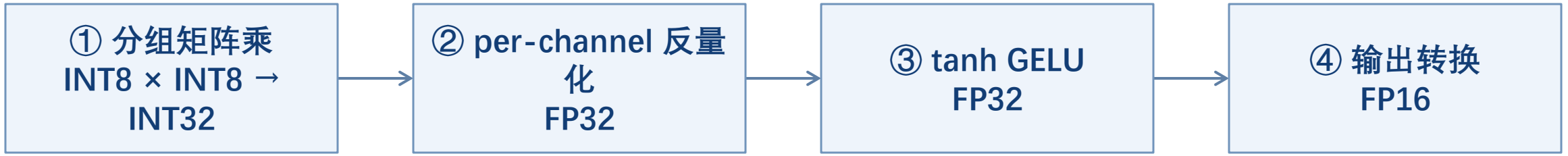


CANN

# （2/3）杭厦决赛赛题解读-题目概述

## ■ 题目概述：

实现 GroupedmatmulGelu 算子需完成：依据grouplist分组->矩阵乘法->反量化->GELU




## ■ 算子功能计算公式

$$C[m,n] = \sum_k x[m,k] \times weight[e,k,n]$$

$$z[m,n] = float(C[m,n]) \times scale[e,n]$$

$$GELU(z) = 0.5z \times [1 + tanh(\sqrt{(2/\pi)} \times (z + 0.044715z^3))]$$

$$y[m,n] = FP16(GELU(z[m,n]))$$

## ■ 分组属性：

group_list 为累计结束位置，例如 [2,2,5] 对应各组长度 [2,0,3]。
scale[e,n] 只沿专家 e 的 token 行广播，空专家跳过计算且索引不变。

CANN

# （3/3）上合决赛赛题解读-约束说明

| 参数名 | 类型 | 形状维度 | 支持数据类型 | 数据格式 | 备注 |
| --- | --- | --- | --- | --- | --- |
| x | 输入 | (M, K) | INT8 | ND | 已按专家排列，固定不转置 |
| weight | 输入 | (E, K, N) | INT8 | ND | 各专家独立权重，固定不转置 |
| scale | 输入 | (E, N) | FP32 | ND | 合并后的 per-channel 反量化因子 |
| group_list | 输入 | (E,) | INT64 | ND | 非递减，末值为 M；允许空专家 |
| y | 输出 | (M, N) | FP16 | ND | GELU 后转换，保持 token 行顺序 |

范围：$1 \le E \le 128$，$1 \le M \le 65536$，$1 \le K \le 8192$，$1 \le N \le 4096$；无属性参数

规模：INT8 权重≤256 MiB；四个输入与最终输出的总字节数≤512 MiB

数值：scale 有限非负，且≤min(1, 60000/(16384K))；零 scale 对应通道输出零

布局：仅 M 轴分组、连续 ND；$M_e$、K、N 可非对齐；GELU 固定 tanh 定义

CANN

# 赛事规则解读

① Host / Kernel 分工：Host 仅负责参数校验、shape 推导、tiling 逻辑和启动配置；Kernel 负责完成完整计算流程。
示例：Host 只计算 tiling / block 数；若 Host 先算好部分结果再传给 Kernel，即违规。

| # | 禁止行为（12 条） | 示例说明 |
|---|---|---|
| 1 | Host 侧计算或重排输入 | Host测预先计算矩阵乘、反量化或 GELU； |
| 2 | 硬编码或逐用例编写专属实现 | 根据用例编号、随机种子或数据规律跳算 |
| 3 | 省略必要计算或改变算子语义 | PerToken：忽略 xScale、截断负值。<br>GELU：替换规定的激活函数，或在 GELU 前将反量化结果转为 FP16。 |
| 4 | Host 侧完成向量/矩阵计算，传入 Kernel 使用 | Host 先算好结果，Kernel 直接用 |
| 5 | 硬编码、按测试用例或数据规律跳算 | 按固定 shape/case_id 直接返回答案，或识别数据规律后跳算 |
| 6 | 写入标记跳过计算 | Kernel 写 flag/标记，后续轮次看到标记就不再计算 |
| 7 | 利用旧输出或未初始化内存跳过计算 | 复用上一次的 y 或未初始化 workspace 当作本次结果 |
| 8 | 逐条匹配评测用例编写专属实现 | 对数据组写单独分支/单独 kernel |
| 9 | 修改或干扰判题、精度校验和计时 | 改计时区间、绕过精度比较或篡改判题结果 |
| 10 | 注入空 kernel、识别预热轮次或把计算移出计时 | warmup 轮不干活、正式计时才算，或在其他地方计算，注入空 kernel占位等。 |
| 11 | 在多轮次的判题中加缓存来减少耗时 | 第 1 轮算完缓存结果，后续同输入直接返回缓存 |
| 12 | 一次计算通过多次 kernel 来完成计算 | 把一次完整的计算过程拆成多个 kernel 来完成 |

③ **最终核验：**比赛结束后进行人工核验和查重；未按规则提交的代码，官方将按违规代码作废成绩。

# 目录

Part 1 赛程安排

Part 2 赛题解读

Part 3 交流答疑

# 交流答疑

# 讨论贴

# Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://atomgit.com/cann